Garden Sprinkler Controller

A dark-mode CustomTkinter desktop app for seven independently named sprinkler zones plus a channel 8 master pump, connected directly to the **Waveshare Modbus POE ETH Relay (B)**. The headless controller runs as a Linux user service, including while logged out. The desktop app connects locally to that service; it never owns the relays.

Setup and launch

Requires Linux with systemd and Python **3.11 or newer**. Only the desktop app requires Tk, CustomTkinter, and a graphical session. The service uses the standard library and does not need a display. On Debian/Ubuntu, install the Python desktop/virtual-environment prerequisites:

```
sudo apt install python3-tk python3-venv
```

From this folder:

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
mkdir -p ~/.config/systemd/user
cp garden-sprinkler.service ~/.config/systemd/user/
loginctl enable-linger "$USER"
systemctl --user daemon-reload
systemctl --user enable --now garden-sprinkler.service
.venv/bin/python app.py
```

Close any older, direct-hardware version of the app before starting the service. The supplied unit expects this project at `~/Documents/garden-sprinkler-controller`; edit its paths if installed elsewhere. Enabling linger may require administrator approval on other machines. Do not move/delete this project or its virtual environment while it is installed.

Useful service commands:

```
systemctl --user status garden-sprinkler.service
journalctl --user -u garden-sprinkler.service -n 50 --no-pager
systemctl --user restart garden-sprinkler.service
systemctl --user disable --now garden-sprinkler.service
```

Linger starts the user service manager at boot, without requiring a login. The enabled service survives logout and restarts after a process failure. Failed startup (for example the network/relay still booting) retries every 15 seconds. Runtime hardware faults latch scheduling OFF rather than repeatedly retrying ON commands: check the equipment/log, then restart the service. Opening, closing, or reconnecting the desktop does not reset outputs. To stop watering, use Rain Pause or STOP ALL, not the window close button. After updating the project, restart the background service before reopening the desktop so both processes use the same settings format.

The PC must remain awake. This installation does not change machine-wide sleep or BIOS/UEFI settings. For recovery after a mains failure the PC must actually power up again: enable "Restore on AC Power Loss"/"AC Back" in firmware if needed. The service cannot turn on a powered-off computer.

To change relay/configuration parameters, use `systemctl --user edit garden-sprinkler.service` and override `ExecStart` (first clear it with an empty `ExecStart=`). The `--config`, `--host`, `--port`, `--slave-id`, and `--timeout` arguments now belong to **service.py**, not app.py. Restart the service afterward. For foreground diagnosis only, stop the installed service before running `.venv/bin/python service.py`; never run a second hardware controller. The desktop accepts `--socket` for an alternate local service endpoint.

The default is `~/.config/garden-sprinkler-controller/settings.json`. Settings are validated and written using atomic replacement. Missing settings start with eight disabled routines. Invalid existing settings produce an error and are never silently overwritten.

Your Waveshare relay

The supplied device-settings screenshot confirms:

Setting	Value
IP address	`192.168.0.99`
TCP port	`4196`
Work mode	TCP Server
Transfer Protocol	None (Modbus RTU-over-TCP, **not** Modbus TCP framing)
Internal serial configuration	115200, 8 data bits, no parity, 1 stop bit

Leave the board settings unchanged. Zones 1-7 map to relays 1-7 (coil addresses 0-6). Relay 8 (coil address 7) is reserved for the master pump.

The manufacturer's protocol documentation shows slave address **1** in its examples. A read-only status request to your relay at `192.168.0.99:4196` confirmed that slave ID **1** responds and all eight relays reported **OFF**. No relay write commands were sent during this check.

First run a **read-only** status check from a computer on the same LAN. This command needs only Python, not Tk or any third-party packages:

```
python3 waveshare_relay.py
```

If address 1 does not respond, optionally discover a responding slave:

```
python3 waveshare_relay.py --discover
```

Discovery only sends function 01 (read eight coil states), tries addresses 1–247, and stops at the first valid response. It does not send writes or change settings. It can take several minutes if the device is not responding. Cancel with Ctrl+C. If no address responds, check network access, relay power, and port/protocol settings; a failed scan does not establish that the slave ID is wrong.

Open the service-connected desktop:

```
.venv/bin/python app.py
```

The service connects to `192.168.0.99`, port `4196`, slave ID `1`. Service startup sends OFF commands to all eight relays and verifies each state, then starts any saved routines that are scheduled now, unless Rain Pause is enabled. A routine can therefore energize a valve immediately on service start when not paused. The **Read Relay Status** button queries and displays the current physical states through the service without changing them.

There is no simulation mode or direct-hardware fallback. If the service is unavailable, the app displays an explicit error. If connection is lost while open, it marks status unknown, disables watering controls and retries its local connection every second. The service may still be watering while disconnected. Request timeouts are not retried: a command may have succeeded before its reply was lost. Status polling reconciles the result. The local Unix socket is private to the user (mode 0600); there is no network control listener. File locks prevent duplicate service owners using the same configuration or relay host/port. Multiple desktop windows are supported; saving stale edits is rejected rather than overwriting another window's changes.

The network adapter in waveshare_relay.py uses RTU CRC16, checks response addresses, functions, CRCs, write echoes, and coil readback. Requests use a fresh TCP connection and a bounded transaction timeout (default 1 second, configurable with `--timeout`, maximum 10). ON commands are never retried or treated as successful after a timeout; fault handling attempts OFF on all channels instead. Coil readback confirms the device's reported logical state, **not** valve movement, water flow, or contact integrity. Other programs and the board's digital inputs can change outputs; use command-compatible, non-linkage relay modes and avoid competing controllers.

On a hardware communication error, the service stops scheduling, attempts OFF on every channel, records the failure in its log and requires a service restart. The app shows the service fault. Output state is labelled unknown when a fault occurs. STOP ALL remains available to retry OFF commands. Requests are synchronous: the GUI may pause during network operations; a failed all-channel OFF sequence can take up to eight transaction timeouts, plus connection overhead. A successful write also requires a separate readback transaction.

Using the controller

- **Daily Schedule**, above the editor, opens a weekday table of every configured garden period sorted by start time. It includes live unsaved edits (clearly labelled), excludes the master pump, and can hide disabled routines. Select a row to open its relay settings. Amber warnings flag identical starts and overlapping periods across enabled relays; adjacent end/start times are not overlaps. These are advisory only and do not block saving: when two enabled, unpaused schedules overlap, both relays can run at the same time. Watering is not queued sequentially; the shared master pump runs for their combined demand. Incomplete/invalid pairs remain visible with an explicit warning and are not fully conflict-checked. Rain Pause and stale service status are labelled; the overview shows configured times, not current relay outputs.
- **Weekly Overview**, beside Daily Schedule, shows every garden relay against all seven days at once. Each day cell lists all four numbered start-stop periods (unused periods show `--`). Bright red text and `!` flag same-start or overlapping periods on both affected relays; red cells flag invalid or incomplete pairs. Click a garden or cell to open its settings. Both windows can remain open together and update as you edit. Both support hiding disabled routines, label unsaved edits, and exclude the master pump. Comparisons are within the same weekday, not across days.
- The compact opening window is 940 x 760, with a minimum size of 940 x 620. One **RUN ZONE / STOP THIS ZONE** button handles timed watering and stopping. STOP ALL and Save Settings stay fixed at the bottom. At

smaller heights, scroll the configuration panel to reach lower fields; the watering controls and Rain Pause remain visible outside that panel.

- Select each garden zone (1-7) on the left, edit its **Relay / Garden Name** at the top of the settings panel (up to 40 characters), then press **Save Settings**. The custom name appears beside that relay in the left-hand list and is saved separately for each relay.
- Enable its routine, then set up to **four independent watering windows for each weekday** in the daily schedule. Matching start/stop pairs use muted blue, green, amber, and purple backgrounds to distinguish the four windows. Enter each start and stop in **24-hour time**, using **HMM** or **HHMM** (for example **815**, **0815**, or **0600**); **HH:MM** such as **08:15** is also accepted. Times display in **HH:MM** format. Leave both fields blank to skip a window; if you enter one time, its matching start or stop is required. Window numbers are independent: enter them in any order, but do not overlap windows within one zone. Windows cannot cross midnight. Click a **Start** heading to pause or resume that numbered window for the selected zone across all weekdays; the smaller **Paused** marker appears below paused headings. Changes take effect when saved. Red borders identify the time fields for overlapping windows. Clicking a conflicting Stop field opens an acknowledgment popup with the conflicting groups. Enabled routines need at least one complete time window; a different weekday can have different times.
- To reuse a day's schedule, choose that weekday in the menu above the schedule and check **Copy selected day's times to all days**. This copies all four windows once; each day remains independently editable afterward.
- Older version 1, 2 and 3 settings are migrated in memory, preserving existing weekdays and times in their original windows and leaving new windows blank. The upgraded version 4 settings are written on the next save.
- Validation updates while typing. **Save Settings** saves and applies **all seven garden zones and the preserved channel 8 settings**. Unsaved edits survive switching between zones but do not alter the running schedule. Closing prompts to save or discard them.
- Red indicates a commanded/confirmed energized relay, bright green an enabled schedule that is active at the current time, and grey no current watering window. Sidebar status uses the **saved/applied** routine, not unsaved edits.
- **RUN ZONE** waters the selected zone for up to **10 minutes**, regardless of whether a routine is enabled, unless Rain Pause is on. Its button becomes **STOP THIS ZONE** whenever that zone is watering.
- **RAIN PAUSE** in the left sidebar immediately sends OFF to all eight relays and blocks scheduled and manual watering until switched off. It saves automatically, independently of unsaved schedule edits, and survives restart. Schedules remain unchanged. Turning it off asks for confirmation: a currently active saved routine may start immediately for its remaining time. Previous STOP THIS ZONE/STOP ALL suppression still applies; an interrupted manual run does not restart. Pause state is stored beside the selected settings file, with `.rain-pause.json` appended to its name (normally `settings.json.rain-pause.json`); include this file in backups. A missing pause file means not paused. An invalid file prevents startup; a save or relay failure displays an error and stops scheduling.
- **STOP THIS ZONE** and **STOP ALL** act immediately and suppress the currently active automatic watering window(s) until they end. Later scheduled windows that day are unaffected. Another manual run can still be started. Suppression is not persisted: restarting the service or PC during a saved window starts that routine again unless Rain Pause is enabled. Closing or reopening the desktop does not clear suppression.
- A manual watering run replaces that zone's current scheduled run; when the timed override ends, the routine does not resume in the same window. Disabling a routine and saving stops its scheduled run, but does not cancel an independent manual run; use **STOP THIS ZONE** for that.

Channel 8 master pump

Channel 8 is labelled **Master Pump**. It automatically switches ON whenever any garden zone (1-7) is running, whether scheduled or manually. It has no independent schedule or watering control. Its red sidebar dot indicates the commanded/confirmed pump ON state.

The app opens the first zone valve before starting the pump. The pump stays ON while other zones are running and stops before the last valve closes. Startup, STOP ALL, Rain Pause, fault shutdown, and normal app closure attempt pump OFF first, then all seven valves. There are no added valve-travel or pump delays; command order is not proof that a physical valve has finished moving. Failed ON commands trigger an all-output OFF attempt and stop scheduling. Communication failures can prevent OFF commands from reaching the device.

The original channel 8 name and schedule are preserved in settings but ignored in pump mode. To restore eight independent garden zones, stop the service and set `MASTER_PUMP_CHANNEL = None` near the top of controller.py, then restart the service. The nearby comment explains this switch; the original scheduling and editing logic remains available, rather than being deleted. Restoring eight-zone mode also restores any enabled saved channel 8 routine, which may start immediately. Review it before restoring that mode.

Use a suitably rated pump contactor where required; do not assume the relay contacts can handle a pump's motor starting current. Have mains wiring and protection checked by a qualified electrician. Software does not replace independent dry-run, pressure, overload, or emergency-stop protection.

Timing and operational limits

The service must be running and the computer awake for schedules to run. It checks outputs every 250 ms when idle; network operations can delay these checks. Start/stop actions do not have real-time precision. It uses the operating system's **local time and weekday**. Starting the service during a scheduled window runs only its remaining time; missed windows are not replayed.

Routine runs end at their stop time or their originally calculated elapsed-time deadline, whichever occurs first. Manual/test durations use a monotonic clock. This prevents a backwards wall-clock adjustment from extending an active run. Clock changes and daylight-saving transitions can shorten or skip watering; this is not a timezone-aware scheduling service. Multiple zones may run simultaneously; the master pump follows their combined demand. There is no sequential queuing or flow/pressure monitoring.

Normal service shutdown sends OFF to all eight outputs. Closing the desktop leaves watering running. A killed service process, network failure, sleeping computer, or frozen event loop cannot guarantee shutdown. **Timers are software-only; they do not program a hardware auto-off or watchdog.** Use normally-open contacts and an independently designed hardware cutoff. Test with valve power disconnected first; verify correct channel mapping, command mode, readback, and shutdown before connecting actual valves. Never rely on the app alone to prevent flooding or protect a pump.

Code layout

The UI and scheduling code use the `RelayBackend` protocol in controller.py.

- app.py: dark-mode desktop service interface.
- service.py: sole hardware owner, headless scheduler and local IPC.
- service_client.py: desktop snapshots and control requests.
- garden-sprinkler.service: boot/logout systemd user unit.
- controller.py: schedule engine, timed overrides, JSON persistence.
- waveshare_relay.py: RTU-over-TCP adapter and read-only status/discovery command.
- test_controller.py and test_waveshare_relay.py, and test_app.py, and test_service.py: automated tests. GUI checks require a display and use mocked relay writes.

Tests

The core and network adapter have no third-party dependencies and can be tested without Tk or a display:

```
python3 -m unittest -v test_controller test_waveshare_relay test_service
```

Tests cover validation, configuration round-trips and failure preservation, schedule boundaries, weekdays, exact manual/test deadlines, override suppression, clock rollback, independent zones, and shutdown behavior. Protocol tests cover vendor CRC examples, fragmented responses, malformed frames, device exceptions, timeout errors, write/readback validation, and read-only discovery. A loopback TCP server tests controller startup, manual operation, and shutdown over real local sockets. Service tests use private local Unix sockets and fake hardware, including logout-safe client closure, exclusive ownership, restart state, stale-edit rejection, Rain Pause and fault latching. Run `.venv/bin/python -m unittest -v` to include GUI checks with a display. Tests do not contact the physical relay board.